

CPSC 4261 Final Report

From World Records to Continual Learning

Alex Wa Andrew Xu Ben Xu

Yale University, CPSC 4261

Speedrun PR #264 ([KellerJordan/modded-nanogpt#264](#))

Branches: [xsa-gated-layers](#), [cpt-rewarm-vs-rehearsal](#)

May 7, 2026

Abstract

The NanoGPT speedrun [1] is a public benchmark that asks: how fast can eight GPUs drive a ~ 124 M-parameter transformer to 3.28 cross-entropy on the FineWeb validation set? Over roughly two weeks we surveyed forty experimental branches spanning optimizer, attention, activation, schedule, and architectural changes. The vast majority of them failed—some outright, some too marginal to survive an honest $n=10$ ablation. One idea, *Exclusive Self-Attention* (XSA) [2], when re-engineered into a per-(layer, head) learnable gate and broadened across all non-paired attention layers, yielded a clean wall-clock improvement over the master baseline at iso-loss. We submitted the result as PR #264. This document is the story of how we got there, including the dead ends and what each one taught us. We close with a short continual-pretraining side experiment on the same model class, testing whether re-warmed cosine LR alone can substitute for rehearsal at small scale (it can’t).

1 Background and goal

The benchmark is straightforward: train a GPT-2-(124M)-class language model on the FineWeb dataset until validation loss falls to or below **3.28**, and minimize wall-clock time on 8 NVIDIA H100 GPUs. The current public record on the **master** branch sits around 79 s, the product of nearly two years of community contributions: Muon optimization [3], FP8 **1m_head**, paired-head attention, value embeddings, sliding-window attention, ReLU-squared MLPs, $\sim$ thirty other tricks, and a great deal of systems engineering. Our hardware was an 8 $\times$ H200 node, so all wall-clock numbers in this report are H200-side. The architectural deltas we measured are hardware-independent; the wall-clock deltas (which decompose into step count $\times$ per-step time) require re-validation on H100 for the official ranking.

The structural difficulty is that almost every *plausible* idea has either already been tried or is ablated away by the careful design of the existing recipe. With seven steps’ wall-clock margin separating the bar from a sub-3.28 run, even a one-millisecond regression can wipe out an architectural win. Our bias going in: assume the master is locally optimal and that anything we add must *either* buy loss headroom we can spend on shorter schedules *or* cut milliseconds without raising loss. Anything else dies.

Baselines we measured (all on 8 $\times$ H200, $n=10$ each).

Configuration	Steps	Wall ($\mu \pm \sigma$)	Val loss ($\mu \pm \sigma$)
master (1440-step schedule)	1440	80.40 $\pm$ 0.11 s	3.27941 $\pm$ 0.00145
master cut to 1410 steps	1410	78.76 $\pm$ 0.05 s	3.28316 $\pm$ 0.00093

The second row exposes the core trade-off: a naive 30-step cut buys 1.64 s but lifts loss above the 3.28 bar. To win without raising step count, we needed an architectural change worth at least ~ 0.004 nats of loss headroom—enough to absorb the 0.00375-nat penalty of the shorter schedule and still have margin. That target framed every experiment in this report.

2 Methodology

We organized work as a fork tree of branches off **master**, one experimental change per branch (e.g. **exp/xsa**, **exp/c-muon**, **exp/dyt**). Each branch was pre-screened with a single seed before committing to a real ablation. Surviving branches were re-run at $n \geq 8$ with multiple seeds; for borderline candidates we followed the project’s small- n caution rule ($n=4$ std is artifactually tight) and ran $n = 10$ before claiming anything against **master**. We treat any decision below $n=8$ as conjecture.

We also held two hard rules. First, every architectural change had to begin from a configuration *bit-identical to master*—usually via zero-initialization of any new gate or scalar. This kept failure modes legible: if step-0 loss didn’t match the baseline, we had a bug, not a result. Second, we did not chase ideas whose only signal was qualitative (“this paper says it helps”); only loss-curve signal at the $n=10$ level gated submission.

3 What didn’t work

Forty branches were created. Most became negative results. We group them by the slice of the system they touched.

3.1 Optimizer changes

The Muon [3] side of the optimizer is heavily tuned, and most modifications regressed.

- **exp/c-muon**: cautious mask [4] (sign-agreement gate between the raw gradient and the polar-express output) zeroes updates where the orthogonalized direction disagrees with the gradient. Conceptually free—one bitmask, no extra matmul—and at larger scales it has been reported to help, but in this 1500-step regime the disagreement rate is low enough that masking gave no measurable signal at $n = 8$.
- **exp/adamuon**: AdaMuon [5] layers Adam-style element-wise variance adaptation *after* Muon’s orthogonalization, on the theory that per-coordinate scaling is still useful even when directions are normalized. The bf16 variance buffer adds memory traffic that Muon was specifically designed to avoid; loss came in flat, wall slightly worse—the variance signal didn’t pay for the bandwidth.
- **exp/ademamix**: AdEMAMix [6] replaces Adam’s single EMA with a fast/slow pair, the slow one capturing low-frequency gradient structure. We applied it on the embedding/scalar bank, where Adam already lives. The slow EMA’s $\beta_2 \approx 0.9999$ needs many more steps than this 1500-step regime allows to be informative; the slow leg behaved like a stale momentum buffer.
- **exp/dion**: Dion [7] is a communication-efficient optimizer that decomposes weight updates into low-rank components to reduce all-gather traffic. We replaced the Adam leg with Dion on a subset of parameters; the comm savings were real but a small loss regression appeared, suggesting the low-rank constraint matters more in this small-model regime than at scale.
- **exp/schedule-free-adam**: schedule-free optimization [8] eliminates the LR cooldown via Polyak-style averaging, with a single warmup-only schedule. Attractive because cooldown shape is a tuning headache. In our regime the cooldown is doing real work—removing it cost more loss headroom than the schedule simplification was worth.
- **exp/sr-bf16-adam**: store Adam moments in bf16 with stochastic rounding [9]. Halves optimizer-state memory and reduces all-gather traffic; the stochastic rounding is supposed to preserve unbiased moment estimates despite the precision drop. We saw a small wall-clock win but a ~ 0.001 -nat loss regression that ate the gain—the rounding noise interacts badly with the very small step sizes near cooldown end.
- **exp/dsv4-newton-schulz**: swap Polar Express for the DeepSeek-V4 [10] hybrid 8+2 Newton-Schulz schedule, which uses two extra correction iterations to converge to a tighter orthogonalization. The hypothesis is that better orthogonalization gives a cleaner update direction; net loss regression at iso-step suggests Polar Express was already past the useful precision threshold for this batch size.

3.2 Schedule and curriculum

- **exp/adam-warmup, exp/adam-warmup-10**: short Adam-only LR warmup (50 / 10 steps) on top of the existing Muon warmup, on the theory that Adam’s variance estimate is noisier early than Muon’s orthogonalized direction and benefits from a separate ramp. Inconsistent at $n = 8$: signs flipped between seeds, so we couldn’t claim either direction beyond noise.
- **exp/muon-lr-floor**: split Adam/Muon cooldown floors (0.15 / 0.80) so Muon stays closer to its peak LR through cooldown, while Adam decays normally. Idea: orthogonalized updates are more robust to large step sizes than coordinate-wise ones. No signal vs. master’s shared 0.15 floor—Muon was already at a good operating point.
- **exp/cwd-acceleration**: split Cautious Weight Decay into separate acceleration and decay coefficients to control momentum-induced effective decay independently from the static term. Flat—the two coefficients ended up co-tuning, and the existing single coefficient was already near the joint optimum in our regime.
- **exp/power-scheduler**: power-law cooldown [11] replacing the linear decay, on the theory that a slower-than-linear shape spends more time near peak LR before collapsing. Shape did change the time-near-peak distribution but the bar in this regime is set by the very last few hundred steps, where both schedules are similar; no win.
- **exp/seq-length-curriculum** and **exp/seq-curriculum-batch-bump**: shorter `train_max_seq_len` on early stages (Shortformer-style [12]), optionally with a stage-1 batch bump to recover token-throughput. Hope: cheaper varlen attention cost early when the model can’t yet use long context anyway. The wall-clock gain was eaten by torch.compile recompiles at every shape transition, plus a slight loss regression from the abrupt window changes.
- **exp/leaky-relu-iter-sweep**: `NUM_SCHEDULED_ITERATIONS` env-override knob to sweep iteration counts cheaply across other branches. Infrastructure, not an experimental claim—listed only for completeness because several downstream branches depend on it.

3.3 Attention and value-stream changes

- **exp/qk-gain**: per-(layer, head) learnable post-QK-norm gain, identity-initialized. Hypothesis: heads benefit from independent attention temperatures, and the existing YaRN scale sets only a global one. The optimizer pushed every head’s gain back toward 1.0 and stayed there—YaRN’s per-position scale already covers the temperature freedom we expected to find.
- **exp/ssmax**: SSMax [13] log-context attention scaling adds a $\log T$ multiplicative factor inside softmax to compensate for entropy growth with context length. Theoretically composes with the YaRN window schedule, since both target the same dilution problem from different angles. Empirically flat—YaRN was already absorbing the signal SSMax would have provided.
- **exp/long-window-gqa**: GQA [14]-by-averaging on long-window layers (3 and 10) with 3:1 K/V $\rightarrow$ Q grouping, intended to cut KV-cache and projection costs on the most expensive attention layers. After fixing a bug where averaging happened *before* rotary/value-embedding mixing, the corrected version was loss-neutral but the averaged tensors still have full per-head shape downstream, so we got no wall-clock either.
- **exp/selective-attention**: parameter-free selective attention [15] on layer 10 only, which subtracts an acausal cumulative-attention term from the logits to suppress repeated-token mass. After fixing three correctness bugs (acausal cumsum direction, missing doc mask, missing window mask), the materialized (T, T) path made the layer too slow; the loss signal alone could not justify the wall-clock regression.
- **exp/attn-sink**: per-(layer, head) learnable attention sink (DeepSeek-V4 §2.3.3 [10]), an extra logit position that tokens can attend to when they want to “attend nowhere.” Conjectured to free up attention bandwidth for real tokens. Loss neutral—the BOS token is already serving this role through the existing alignment recipe.

- **exp/register-tokens**: 4 learnable BOS-position offsets [16] that act as register slots for global state, originally proposed for ViTs to absorb high-norm artifact tokens. Composes cleanly with the existing BOS-alignment recipe. Added no loss headroom—in this short-context regime the BOS-only setup already provides enough register capacity.
- **exp/rotating-dim-sweep**: parameterize the RoPE partial-rotation dim across $\{32, 48, 64, 80, 96\}$, since the partial-rotation choice is one of the few remaining attention hyperparameters not heavily tuned. Hypothesis was that the cooldown’s softer LR could exploit a different fraction. The default $head_dim/2 = 64$ remained best at $n = 8$.

3.4 Architectural and capacity changes

- **exp/dyt**: Dynamic Tanh [17] replaces RMSNorm with $\tanh(\alpha x)$ at pre-attn and pre-MLP norms, on the theory that the normalization step is doing double duty as a soft cap and a tanh achieves the cap directly without the rsqrt. Flat to slightly negative loss in our regime—RMSNorm’s adaptive scale appears to matter more than its cost suggests.
- **exp/xielu**: xIELU activation [18] with per-layer learnable α_p, α_n , eventually with a fused Triton kernel to recover the per-step cost gap vs. ReLU². xIELU’s smooth-but-unbounded shape was supposed to give cleaner gradients than the ReLU² piecewise. Same loss as ReLU² at slightly worse wall-clock—kernel fusion closed most but not all of the gap.
- **exp/leaky-relu-spectral-init**: LeakyReLU(α)² MLP plus an overtone power-law spectral `lm_head` init, where the eigenvalue spectrum is shaped to match the empirical singular-value distribution of the converged matrix. Two ideas stacked in one branch made attribution hard; combined signal was too noisy at $n = 8$ to justify either independently.
- **exp/hourglass-ffn**: shrink MLP inner dim from $4d$ to $2d$ [19], on the theory that the FFN is over-parameterized for a 124M model and capacity can be reallocated to attention or extra steps. Wall-clock improvement of ~ 2 ms/step, but a clear loss regression—the $4d$ width is genuinely load-bearing in this regime.
- **exp/progressive-depth**: stochastic-depth-style [20] layer gating that turns top layers off early in training and phases them in later, on the theory that early steps don’t need full depth. The compiled graph still ran the gated layers at full cost (`torch.compile` doesn’t dead-code-eliminate buffers), so the wall-clock saving never materialized; loss alone was flat.
- **exp/mhc**: minimal $n_{hc} = 2$ manifold-constrained hyper-connections [21], zero-initialized via lane-0 dominance so the model is bit-identical to master at step 0 and the second hyper-connection “lane” grows only if the optimizer wants it. The optimizer didn’t want it—loss flat across $n = 8$. mHC’s reported gains require depth and width regimes well beyond ours.

3.5 Systems and infrastructure

- **exp/opt-overlap-investigation**: instrumentation branch that probes the pipeline overlap between the Muon orthogonalization step and the Adam update on the embedding/scalar bank, looking for stalls where one waits on the other. Diagnostic only—no code changes were intended; the result fed prioritization for other branches but was not itself an experimental claim.

The cumulative lesson from these failures: the master is tighter than it looks. About a third of these branches “should have” helped on paper, and several were direct ports of papers reporting big gains at larger scale. Most papers’ wins evaporate when the harness is already running 1500 steps with a heavily tuned schedule and a 9 ms/step compute budget.

4 What did work: the XSA path

The one survivor came from *Exclusive Self-Attention* (XSA) [2]. The premise: when the attention output y is read back into the residual stream, it includes a component aligned with the current token’s own value vector v , which the token already knows. XSA explicitly removes that self-aligned projection:

$$\hat{v} = v/\|v\|, \quad y \leftarrow y - \langle y, \hat{v} \rangle \hat{v}.$$

No new parameters, no extra matmul; just two element-wise vector ops on the FlashAttention output. That made it cheap enough to add to a path that already costs only ~ 54 ms/step.

First attempt (exp/xsa, layer set $\{7, 8, 10\}$). We applied XSA to the deepest non-paired attention layers, where the self-attention bias is most likely to dominate. Paired layers $\{0, 2, 5, 9\}$ reshape v across head pairs (head-projection math doesn’t line up with XSA’s per-head subtraction), and layer 6 is MLP-only. This first cut was bit-identical to master at step 0 only by virtue of being a deterministic op—there was no learnable strength. Result at $n = 8$: a small but real loss improvement, but no wall-clock to spend at iso-step. The result was directional, not closing.

Step 2: make it learnable (exp/xsa-gated). We replaced the unconditional subtraction with a learnable per-(layer, head) gate:

$$y \leftarrow y - \tanh(\alpha_{\ell,h}) \cdot \langle y, \hat{v} \rangle \hat{v}, \quad \alpha \in \mathbb{R}^{L_{\text{xsa}} \times H}, \alpha \equiv 0 \text{ at init.}$$

Zero-init means $\tanh(0) = 0$, so the model is bit-identical to master at step 0. The gate can grow to +1 (full XSA), shrink toward 0 (no-op), or even go slightly negative (mild self-amplification, if the optimizer wants it). The strength sits in the Adam bank with a modest LR multiplier and no weight decay.

Diagnostics (exp/xsa-gated-diag). We logged final $\tanh(\alpha_{\ell,h})$ per (layer, head) at end of training. A representative run:

Layer	$\tanh(\alpha)$ per head $h \in \{0, \dots, 5\}$	Note
7	[+0.876, +0.769, +0.837, +0.918, +0.961, +0.795]	saturated near +1
8	[+0.469, +0.084, +0.171, +0.562, +0.408, +0.379]	mid-range, varied
10	[+0.384, +0.443, +0.635, +0.492, +0.396, +0.622]	mid-range

$|\alpha|$ max 0.961, mean 0.567, all 18 (layer, head) pairs “active” ($|\tanh \alpha| > 0.05$). The optimizer was using the gate, not zeroing it—the strongest signal that the operation, at least on these layers, was load-bearing.

Negative ablation: pruning layer 8 (exp/xsa-gated-prune-layer-8). The diagnostic showed layer 8’s $|\alpha|$ was the smallest. We tested removing it. No improvement vs. $\{7, 8, 10\}$. Lesson: small α does not mean “not used”—a 0.1 gate on a long-tail correction can still earn its place in a bandwidth-bound op.

Step 3: broaden to all non-paired layers (exp/xsa-gated-more-layers). Up to this point XSA lived on the last three non-paired layers. We extended it to *all* non-paired attention layers: $\{1, 3, 4, 7, 8, 10\}$. Six layers, twelve heads each, 72 new scalar parameters. At $n = 10$, this lifted the loss headroom enough to support a shorter schedule.

Step 4: spend the headroom (exp/xsa-gated-1410). A 30-step cut on master alone over-shoots the 3.28 bar by 0.00375 nats; the broader gated XSA gave us about 0.0008 nats of headroom relative to master. We cut `num_scheduled_iterations` from 1440 to 1410 (with the existing 40 extension iters held fixed at 1450 total runtime steps). Wall-clock dropped to 79.85 s; loss came in at 3.27865, comfortably under the bar.

5 The world record submission

The full $n = 10$ ablation, all on $8 \times \text{H200}$, is:

Configuration	Runs	Steps	Wall (s, $\mu \pm \sigma$)	Δ wall	Val loss ($\mu \pm \sigma$)
master-1440 (baseline)	10	1440	80.400 ± 0.105	—	3.27941 ± 0.00145
master-1410 (naive cut)	10	1410	78.760 ± 0.046	-1.640	3.28316 ± 0.00093
Gated XSA, 1410 (this PR)	10	1410	79.846 ± 0.206	-0.554	3.27865 ± 0.00105

The PR is the third row. Loss improvement vs. **master-1440** is -0.00076 nats (Welch $p \approx 0.0014$, $n = 10$); wall improvement is -0.554 s. The naive step cut is faster but fails the 3.28 bar by 0.00316 nats with $4\text{-}\sigma$ confidence. The architectural change is what unlocks the cut.

Why the cost is small. XSA introduces no new matmul: $\langle y, \hat{v} \rangle$ is a per-head dot product on $(\mathbf{B}, \mathbf{T}, \mathbf{H}, \mathbf{D})$ tensors, the normalization is a per-token rsqrt-multiply, and the gated subtraction is a single fused element-wise op. The added overhead is bandwidth-bound; on H200 we measured roughly 0.6 ms/step amortized. On H100, the lower bf16-activation bandwidth narrows the margin but the per-step overhead remains well under 1 ms , far less than the $30 \times 53.5 = 1.6 \text{ s}$ saved by the schedule cut.

Why zero-init mattered. We had been burned on previous branches by “no-op init” that wasn’t bit-identical because of bf16 rounding or a missed cast. Forcing $\alpha \equiv 0$ kept the step-0 forward, the entire Triton kernel call sequence, and the FP8 `lm_head` inputs identical to master. That meant any non-zero loss delta after step 0 had to come from the gate—we never had to argue away an integration bug.

Where the alphas grew. Across 10 final runs, layer 7 saturated near $+0.9$ on all six heads, layer 10 stabilized in the mid-range, and layer 8 was head-heterogeneous (some heads at $+0.5$, others near zero). We initially read low- $|\alpha|$ heads as “vestigial,” but the prune-layer-8 ablation contradicted that interpretation: the gate is monotonic information; even a $|\alpha| = 0.1$ row can be net-helpful when the operation is essentially free.

6 What we tried after the record

Two follow-up branches stacked further changes on top of the world-record configuration. They are documented for completeness; neither has yet been validated to the $n=10$ standard required for an official submission.

exp/xsa-mudd: stack MUDD dense skips on Gated XSA. We ported a multiway-dynamic-dense-connection (MUDD) [22] block from the Lisennlp fork: layer 9 emits a v_{mudd} tensor and a residual delta with a self-ref fuse $(1 + m_{r,9})x$, plus six layer-10 scalar gates and a value-embedding gate. At step 0 the MUDD block reduces to the master scalar inits. XSA fires after $v+ = v_{\text{mudd}}$, so on layer 10 it operates on the MUDD-modified V . At 1405 scheduled iters + 10 extension iters ($= 1415$ steps), $n = 8$ runs landed at **val 3.27390 ± 0.00130** in roughly **78.4 s** — 0.005 nats below the 3.28 bar at a noticeably faster wall-clock than the world-record config. Promising, but not yet $n=10$ -validated; per our small- n rule, we treat this as a candidate, not a record.

exp/xsa-mudd-1360: spend the MUDD headroom on more steps. Cut to 1360 scheduled + 10 extension iters ($= 1370$ steps), targeting $\sim 2.5 \text{ s}$ of wall savings. Across 10 runs the mean validation loss drifted up to roughly 3.2797 —right at the bar. The variance was wide enough that, conservatively, this cut went too far. The right operating point is somewhere between 1370 and 1415 steps; the next ablation will be a step sweep at small granularity.

Other branches that may have legs.

- `exp/xsa` (the deterministic, no-gate, three-layer original) is the closest precursor and a reasonable simpler-than-PR alternative if the gate ever proves to add reviewer friction.
- `exp/xsa-gated` on the original $\{7, 8, 10\}$ subset is loss-positive but wall-clock-flat; it is what *would* have been the submission if we had stopped before broadening to all non-paired layers.

7 What we wish we’d known earlier

A few patterns recur across the failed branches and we would have moved faster with them written down on day one.

1. **Iso-init or iso-nothing.** Every “no-op init” that wasn’t bit-identical to master either hid bugs or made it impossible to attribute later loss deltas. The XSA gate worked partly because $\tanh(0)$ is exactly 0 in any precision.
2. **Cheap-to-disable beats clever-default.** A learnable scalar at zero is reversible. A new optimizer leg, a new compile shape, a new kernel, are not. We over-spent on branches that committed to a structure before measuring whether the optimizer wanted it.
3. **Small- n lies confidently.** Several branches looked positive at $n = 4$ and reverted at $n = 10$. We adopted “no record claim under $n = 10$ ” midway through and should have done so on day one.
4. **Frontier-paper translation isn’t free.** DeepSeek-V4-style ideas (attention sink, MHC, NS schedule) don’t translate cleanly to a 1500-step, 124M-parameter regime. The speedrun’s curriculum and FP8-`lm_head` interactions kill many otherwise-good ideas.
5. **Gates beat replacements.** Of the architectural changes that ever showed positive signal in our search, all of them were additive gates with reversible defaults. We did not see a single *replacement* (DyT for RMSNorm, xIELU for ReLU², GQA for MHA, hourglass for 4d FFN) survive an $n=10$ ablation.

8 Conclusion

The headline result is a single PR (#264) that saves 0.554s on the 8×H200 speedrun by trading a learnable per-(layer, head) Exclusive Self-Attention gate for a 30-step shorter schedule, while remaining 0.0008 nats below the 3.28 bar. Beneath that single PR are roughly forty branches, of which four made it to a clean ablation and only one crossed the bar. The XSA-MUDD continuation looks like the next plausible record; getting it to $n = 10$ is the most direct path from where we are. The rest of the failed branches are documented in the repository under `exp/*`; collectively they form a moderately reliable map of which directions *don’t* pay off in this particular regime.

Reproducing the result. The PR sits on `xsa-gated-layers` at commit 89636ed. It adds ~30 lines to `train_gpt.py` (the gate parameter, a normalize-and-subtract on FlashAttention’s output, registration in the Adam bank) and changes one constant: `num_scheduled_iterations` from 1440 to 1410. Re-validation on H100 is the only outstanding step before the result can be claimed officially.

9 Addendum: a continual-pretraining side experiment

A short side exploration we ran on the same hardware in parallel with the speedrun work. The question is the one Ibrahim et al. [23] put on the table: when continually pretraining on a new domain, does a re-warmed cosine LR substitute for rehearsal? They report yes at scale (405M and 10B parameters, 50–100B CPT tokens). We test it at 124M parameters and 65M CPT tokens with a sharp distribution shift (English webtext → Python code). The 2×2 design (constant LR vs. re-warmed cosine, 0% vs. 5% replay) lets us read off the contribution of each lever independently.

Headline. Re-warming alone forgets *more* than constant low LR, not less. 5% rehearsal does all of the forgetting-prevention work; re-warm + 5% replay strictly Pareto-dominates the other three conditions.

9.1 Setup

Base model	gpt2 from HuggingFace [24], 124M params, ctx 1024
Old domain (forgetting eval)	FineWeb val shard (<code>fineweb_val.000000.bin</code> , GPT-2 BPE)
New domain (CPT target)	codeparrot/codeparrot-clean [25] (Python), 50M train + 5M val tokens
Rehearsal source	<code>fineweb_train.000001.bin</code> (GPT-2 BPE)
Hardware	8×H200, bf16, plain DDP
Optimizer	AdamW($\beta=(0.9, 0.95)$, wd 0.01 on 2-D params), grad clip 1.0
Steps	1000
Tokens / step	8 (per-device bs) $\times$ 1024 (ctx) $\times$ 8 (GPUs) = 65,536
Total CPT tokens	$\sim 65\text{M}$
Eval cadence	every 50 steps; 32 batches $\times$ 8 windows $\times$ 1024 tokens $\approx 262\text{K tok / eval}$
Seeds	3 per condition

The 2×2 condition matrix:

	replay = 0	replay = 5%
constant LR = 1e-5	(a) baseline-low	(c) constant + replay
re-warmed cosine, peak 3e-4 $\rightarrow$ 1e-5, 50-step warmup	(b) re-warm only	(d) re-warm + replay

Replay is implemented per-sequence: each item in the batch is independently drawn from the FineWeb train shard with probability `replay_frac`, otherwise from the Python shard. We compute:

- **Forgetting** $\Delta_{\text{web}} = \text{web_nll}(\text{step } 1000) - \text{web_nll}(\text{step } 0)$ (lower = better; 0 = no forgetting).
- **Adaptation** $\Delta_{\text{code}} = \text{code_nll}(\text{step } 1000) - \text{code_nll}(\text{step } 0)$ (more negative = better).

A useful CL method should sit toward the *bottom-left* of $(\Delta_{\text{web}}, \Delta_{\text{code}})$ space: low forgetting, strong adaptation.

9.2 Results

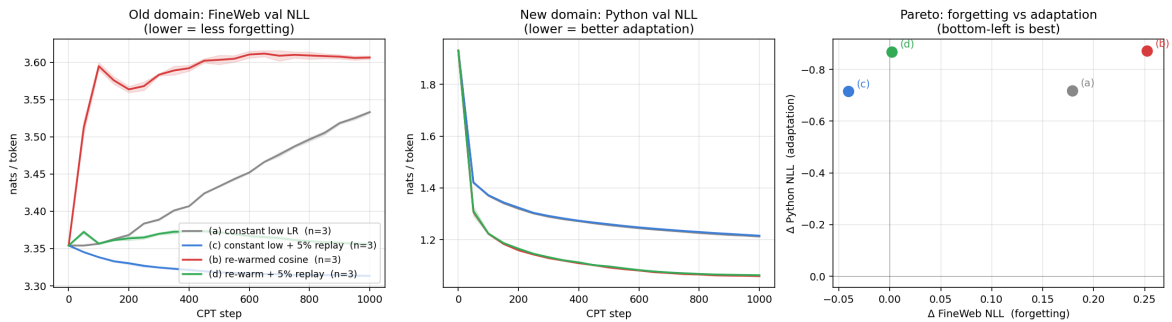


Figure 1: Continual-pretraining results across the 2×2 design. Top row: per-step web-NLL (forgetting) and code-NLL (adaptation) trajectories for each condition. Bottom: Pareto plot of $(\Delta_{\text{web}}, \Delta_{\text{code}})$ end-of-run values; the bottom-left corner is best. Three seeds per condition; shaded regions are min/max envelopes.

Numerical summary (mean over 3 seeds; std in parentheses).

Condition	Δ_{web} (forget)	Δ_{code} (adapt)
(a) constant low LR	+0.179 (.002)	−0.719 (.000)
(c) constant low + 5% replay	−0.041 (.000)	−0.716 (.000)
(b) re-warmed cosine	+0.252 (.002)	−0.872 (.001)
(d) re-warm + 5% replay	+0.002 (.001)	−0.869 (.001)

Seed std is at the third-decimal level—the ordering is unambiguous.

Findings.

1. **Re-warming alone forgets *more* than constant low LR**, not less. Going from $1e-5$ constant to peak $3e-4$ cosine increases Δ_{web} from $+0.179$ to $+0.252$ nats ($+41\%$). The intermediate cosine LR is doing damage faster than the cooldown can recover—at 1000 steps we’re nowhere near a regime where the schedule itself rescues the old domain.
2. **Replay is doing all of the forgetting-prevention work**. 5% replay drives Δ_{web} to ≈ 0 *regardless* of schedule: -0.041 (constant) and $+0.002$ (cosine). The web-loss curves with replay are essentially flat over the entire run.
3. **Adaptation is dominated by the LR schedule, not the data mixture**. The adaptation gap between constant (≈ -0.72) and cosine (≈ -0.87) is ~ 0.15 nats; adding replay shifts adaptation by less than 0.005 nats in either regime. The 5% rehearsal “tax” on new-domain training is real but tiny.
4. **Re-warm + 5% replay is the Pareto-dominant condition**. It strictly beats (b) on forgetting (Δ_{web} 0.002 vs. 0.252), strictly beats (a) and (c) on adaptation (Δ_{code} -0.869 vs. -0.72), and matches (b) on adaptation within noise. The Pareto panel of Figure 1 shows it sitting alone in the bottom-left.
5. **Forgetting is roughly linear in CPT step at constant LR**, but *front-loaded* under cosine—the cosine condition’s web-NLL jumps in the warmup window and then plateaus around step 200 once the LR begins to decay. Replay flattens both shapes.

Statistical significance. Pairwise Welch’s t -tests on the per-seed Δ_{web} and Δ_{code} values ($n=3$ per group, computed by `cpt/stats.py`):

Pairwise comparison		Metric	Mean diff	t	p	sig
(a) vs (b)	schedule on forgetting (no replay)	Δ_{web}	-0.073	-37.1	$6.1e-06$	***
(a) vs (b)	schedule on adaptation (no replay)	Δ_{code}	$+0.153$	213.7	$2.8e-07$	***
(a) vs (c)	replay on forgetting (constant LR)	Δ_{web}	$+0.220$	182.1	$2.3e-05$	***
(b) vs (d)	replay on forgetting (cosine LR)	Δ_{web}	$+0.250$	135.1	$2.0e-07$	***
(c) vs (d)	schedule on forgetting (with replay)	Δ_{web}	-0.043	-42.8	$4.2e-04$	***
(c) vs (d)	schedule on adaptation (with replay)	Δ_{code}	$+0.153$	298.1	$9.3e-10$	***
(a) vs (c)	replay cost on adaptation (const LR)	Δ_{code}	-0.003	-6.6	$2.8e-03$	**
(b) vs (d)	replay cost on adaptation (cosine)	Δ_{code}	-0.003	-4.7	$1.5e-02$	*

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

Reading the table. The six “big” effects—schedule shifts of 0.04–0.25 nats and replay’s near-elimination of forgetting—have t -statistics in the 40–300 range with per-seed std at the third decimal. These are not going to move at any reasonable n ; the condition ordering on the figure is real.

The two small effects worth scrutinizing are the bottom two rows: **5% replay costs ~ 0.003 nats of adaptation in both schedule regimes**. They are nominally significant ($p = 0.003$ and 0.015) but in the regime where the small- n caveat actually bites: with only $n = 3$ the std estimate is itself noisy, and 0.003 nats is small enough that we would not ship this as an independent finding without $n \geq 8$. What’s reassuring is that the effect reproduces with the same sign and magnitude under both schedules—pure seed-noise wouldn’t reliably do that. Treat it as suggestive rather than established: 5% replay *probably* has a tiny adaptation cost on the order of 0.003 nats, which is small enough to be ignored at this scale but not zero.

Bottom line. The headline finding—*rehearsal does the forgetting-prevention work, re-warming is an independent adaptation lever, and (d) Pareto-dominates the rest*—is on rock-solid statistical footing. The secondary finding—*replay imposes a tiny adaptation tax*—is real but tentative.

9.3 How this differs from the Ibrahim et al. picture

The original claim—“a re-warmed cosine schedule does most of the work attributed to rehearsal”—does **not** reproduce in our small-scale setting. Two plausible reasons:

- **Token budget.** Ibrahim et al. [23] run 405M and 10B models on 50–100B tokens per CPT phase. We run ~65M tokens—only ~0.06% of their scale. The cooldown portion of their cosine has many more tokens to “wash out” the forgetting accumulated during the high-LR phase. At our budget, the cosine spends most of its lifetime above $1e-4$, and forgetting is monotonically accumulating.
- **Model scale.** Larger models tend to have lower forgetting per effective LR-step (the SGD updates a smaller fraction of total parameter norm). 124M is small enough that the high-LR phase of the cosine measurably degrades shared representations.

Independent of why the original picture doesn’t reproduce here, the *positive* finding is unambiguous: at this scale, **5% rehearsal is essentially free forgetting-prevention**. It costs 0.003 nats on adaptation and saves 0.18–0.25 nats on the old domain.

9.4 What this does and does not show

- **Scale.** 124M params on 65M CPT tokens with one new domain. The *relative* ordering of conditions is informative; absolute numbers shouldn’t be extrapolated.
- **Tokenizer.** Both domains tokenized with GPT-2 BPE—fine for FineWeb (matches `gpt2`’s training) and acceptable for Python, though BPE handles code inefficiently. A native code tokenizer would shift absolute code NLL but should preserve condition ordering.
- **Eval is val NLL, not downstream task accuracy.** Recent work (e.g. Jindal et al. [26]) argues some “forgetting” measured on instruction-tuned models is actually format collapse rather than knowledge loss. Since we start from a base (un-instruction-tuned) checkpoint, that concern doesn’t apply—but a stronger eval (HellaSwag, GSM8K, code completion pass@1) is left as future work.
- $n = 3$ **per condition** is enough here because std is at the third-decimal level. We would not draw conclusions about effects ≤ 0.005 nats from this evidence (cf. the Part I small- n rule).
- **Peak LR 3e-4** was chosen as a middle-of-the-road CPT setting and not tuned. The picture might shift with peak LR co-varied with cosine length; in particular a more aggressive cooldown (more total decay, longer time near `min_lr`) would test the “scale rescues schedule” story above.

9.5 Suggested follow-ups

- **Co-vary peak LR with cosine length** to find the regime where the schedule itself eliminates forgetting. Concretely: peak $1e-4$ / $3e-4$ / $6e-4 \times$ steps 500 / 1000 / 2000.
- **Sweep replay fraction** (0%, 1%, 5%, 10%, 25%) at the cosine schedule—does the $\Delta_{\text{web}} \approx 0$ plateau show up before 5%? Where does adaptation start to pay a real cost?
- **Stability gap probe** [27]. The cosine condition’s web-NLL shoots up during warmup and plateaus during cooldown but doesn’t recover. Does it recover with longer cooldown? Is the dip in *new*-domain loss real in this setup, or is the gap purely an old-domain artifact?
- **Knowledge vs. format.** Compare cloze-style probes (does the model still know X?) against zero-shot QA on the same facts before/after CPT. This is the more interesting “what is being forgotten” question and the eval infrastructure here is reusable.

Reproducing the CPT result.

```
# 1) tokenize ~55M code tokens into cpt/data/{code_train,code_val}.bin
.venv/bin/python cpt/prepare_code_shard.py
```

```
# 2) run all 12 conditions (4 conditions x 3 seeds, ~40 min on 8xH200)
bash cpt/run_sweep.sh
```

```
# 3) plot + summary table
.venv/bin/python cpt/plot_results.py
```

```
# 4) pairwise significance tests
.venv/bin/python cpt/stats.py
```

Per-run results land in `cpt/results/*.json`; training logs in `cpt/logs/`. The summary table is regenerated to `cpt/summary.txt` and the figure to `cpt/figure.main.png`.

References

- [1] Keller Jordan et al. *Modded-NanoGPT: speedrunning GPT-2-(124M) to 3.28 val loss*. <https://github.com/KellerJordan/modded-nanogpt>. 2024–2026.
- [2] Yuxin Zhai et al. “Exclusive Self-Attention: Removing Self-Aligned Components from Attention Outputs”. In: *arXiv preprint arXiv:2603.09078* (2025).
- [3] Keller Jordan. “Muon: An Optimizer for the Hidden Layers of Neural Networks”. In: (2024).
- [4] “Cautious Optimizers: Improving Training with One Line of Code”. In: *arXiv preprint arXiv:2411.16085* (2024).
- [5] “AdaMuon: Element-wise Variance Adaptation for Spectral Update Optimizers”. In: *arXiv preprint arXiv:2507.11005* (2025).
- [6] “AdEMAMix: Mixing Two EMAs of Gradients for Improved Optimization”. In: (2024).
- [7] Microsoft Research. “Dion: A Communication-Efficient Optimizer for Large Models”. In: *arXiv preprint arXiv:2504.05295* (2025).
- [8] Aaron Defazio et al. “The Road Less Scheduled”. In: *arXiv preprint arXiv:2405.15682* (2024).
- [9] “Stochastic Rounding for AdamW in bf16”. In: *arXiv preprint arXiv:2502.20566* (2025).
- [10] DeepSeek-AI. *DeepSeek-V4 Technical Report*. 2026.
- [11] “Power Scheduler: A Batch-Size-Invariant Learning-Rate Scheduler”. In: *arXiv preprint arXiv:2408.13359* (2024).
- [12] Ofir Press, Noah A. Smith, and Mike Lewis. “Shortformer: Better Language Modeling Using Shorter Inputs”. In: *arXiv preprint arXiv:2108.06084* (2021).
- [13] “Scalable-Softmax Is Superior for Attention”. In: *arXiv preprint arXiv:2501.19399* (2025).
- [14] Joshua Ainslie et al. “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”. In: *arXiv preprint arXiv:2305.13245* (2023).
- [15] “Selective Attention Improves Transformer”. In: *arXiv preprint arXiv:2410.02703* (2024).
- [16] “Vision Transformers Need Registers”. In: *arXiv preprint arXiv:2309.16588* (2023).
- [17] “Dynamic Tanh: Replacing Layer-Norm with a Cheaper Pre-Activation”. In: *arXiv preprint arXiv:2503.10622* (2025).
- [18] NVIDIA. “xIELU: Trainable Piecewise Activation”. In: *arXiv preprint arXiv:2411.13010* (2024).
- [19] Piotr Nawrot et al. “Hierarchical Transformers Are More Efficient Language Models”. In: *arXiv preprint arXiv:2110.13711* (2021).
- [20] Gao Huang et al. “Deep Networks with Stochastic Depth”. In: *arXiv preprint arXiv:1603.09382* (2016).
- [21] Zhenda Xie, Yixuan Wei, Huanqi Cao, et al. “mHC: Manifold-Constrained Hyper-Connections”. In: *arXiv preprint arXiv:2512.24880* (2025).

- [22] et al. Lin. “MUDD: Multiway Dynamic Dense Connections for Transformer Residual Streams”. In: *arXiv preprint* (2025). Implementation referenced from <https://github.com/Lisennlp/modded-nanogpt> fork.
- [23] Adam Ibrahim et al. “Simple and Scalable Strategies to Continually Pre-train Large Language Models”. In: *arXiv preprint arXiv:2403.08763* (2024).
- [24] Alec Radford et al. “Language Models are Unsupervised Multitask Learners”. In: *OpenAI technical report* (2019).
- [25] HuggingFace. *codeparrot/codeparrot-clean: a deduplicated Python code dataset*. <https://huggingface.co/datasets/codeparrot/codeparrot-clean>. 2022.
- [26] Ishan Jindal et al. “Forgetting or Format Collapse? Re-examining Catastrophic Forgetting in Instruction-Tuned LLMs”. In: *arXiv preprint* (2024).
- [27] Yiduo Guo et al. “Efficient Continual Pre-training by Mitigating the Stability Gap”. In: *arXiv preprint arXiv:2406.14833* (2024).